

FamilyTree Al-Nammari • منصة الأنساب ✱

تقرير فحص وتحليل الأداء

# تقرير الفحص الشامل للأداء

تحليل معمق لبنية التطبيق وواجهات ال API والعمليات  
الحسابية في الذاكرة – مع خطة إصلاح مرحلية

تاريخ الإصدار

٦ سبتمبر ٢٠٢٦

المخرجات

١٣ مشكلة حرجية + ٦ مراحل إصلاح

نطاق الفحص

كامل الواجهة الخلفية + ال

Frontend

## فهرس المحتويات

|    |                                                      |    |
|----|------------------------------------------------------|----|
| ١  | الملخص التنفيذي                                      | ٣  |
| ٢  | المنهجية والمعمارية المُستنتجة                       | ٤  |
| ٣  | جدول المشاكل الحرجة                                  | ٥  |
| ٤  | السيناريو الأول: ٥٠-١٠٠ مستخدم متزامن                | ٦  |
| ٦  | المشكلة ١: حجم ال Connection Pool                    | ٦  |
| ٧  | المشكلة ٢: كتابة داخل طلب قراءة                      | ٧  |
| ٨  | المشكلة ٣: لا فهارس قاعدة بيانات                     | ٨  |
| ٩  | المشكلة ٤: التحقق من الهوية (Auth)                   | ٩  |
| ١٠ | المشكلة ٥: إيميلات متسلسلة + إحصائيات + مزامنة       | ١٠ |
| ٥  | السيناريو الثاني: ٢٠ ألف سجل في الشجرة               | ١١ |
| ١١ | المشكلة ١: تحميل كل السجلات في كل طلب                | ١١ |
| ١٢ | المشكلة ٢: N+1 في الزيجات + Levenshtein في ال dedup  | ١٢ |
| ١٣ | المشكلة ٣: خوارزميات $O(N^2)$ JS في kinship + layout | ١٣ |
| ١٤ | المشكلة ٤: انهيار المتصفح عند ٢٠ ألف عقدة            | ١٤ |
| ٦  | خطة الإصلاح المرحلية                                 | ١٥ |
| ١٥ | P0 – Quick Wins · يوم واحد                           | ١٥ |
| ١٦ | P1 – Database Indexes · يوم واحد                     | ١٦ |
| ١٧ | P2 – Pagination + Caching · أسبوع                    | ١٧ |
| ١٨ | P3 – Server Refactoring · أسبوعان                    | ١٨ |
| ١٩ | P4 – Client Optimization · أسبوع                     | ١٩ |
| ٢٠ | P5 – Monitoring + Hardening · مستمر                  | ٢٠ |

٢١

الجدول الزمني والنتائج المتوقعة

٧

٢٢

الخلاصة والتوصيات

٨

# الملخص التنفيذي

فحص شامل لكود `src/` و `drizzle/` في مشروع **FamilyTree AI-Nammari** (Next.js 14 + Drizzle ORM + PostgreSQL عبر Supabase Pooler). كل مشكلة في هذا التقرير مرفقة بـ `file:line` فعلي من الكود الحالي، ولا توجد أي تكهنات أو تقديرات غير مبنية على قراءة الكود.

## المكدّس التقني المُكتشف

| الطبقة         | التقنية                                     | الدليل في الكود                                       |
|----------------|---------------------------------------------|-------------------------------------------------------|
| إطار الواجهة   | Next.js 14 App Router                       | package.json:27 • next@14.2.18                        |
| ORM            | Drizzle ORM 0.45                            | src/db/index.ts:1,17                                  |
| قاعدة البيانات | PostgreSQL عبر 6543 Supabase Pooler         | src/db/index.ts:5,11                                  |
| المصادقة       | Supabase Auth + JWT Bearer                  | src/lib/supabase/auth.ts:20-21                        |
| تخطيط الشجرة   | Dagre + @xyflow/react                       | src/lib/layout.ts:1 • package.json:17,20              |
| الاستضافة      | VPS Linux + PM2 + Nginx (Long-Running Node) | ecosystem.config.cjs • deploy.sh • nginx.conf.example |

## المؤشرات الحرجة بالأرقام

|                                              |                                           |                                     |                              |
|----------------------------------------------|-------------------------------------------|-------------------------------------|------------------------------|
| ١٠٠+<br>ثانية لاستجابة ١٠٠ طلب متزامن حاليًا | ♦<br>فهارس (Index) على جدول relationships | ٥<br>اتصالات DB كحد أقصى في ال Pool | ١٣<br>مشكلة حرجة/عالية موثقة |
|----------------------------------------------|-------------------------------------------|-------------------------------------|------------------------------|

### 🔴 النتيجة الرئيسية (بدون إصلاح)

التطبيق يعمل بحالة مقبولة لمئة أو بضعة مئات من السجلات وعشرات المستخدمين، لكنه ينهار حتميًا عند ٢٠ ألف سجل أو ١٠٠ مستخدم متزامن. نقطة الانهيار مُحَقَّقة فعليًا في الكود (لا تحتاج تجارب ضغط لتأكيدها).

## التوصية المرحلية

خطة من ٦ مراحل متسلسلة. تبدأ بتغييرات صغيرة منخفضة المخاطر (يوم واحد) وتنتهي بإعادة هيكلة معمارية (أسبوعان). المرحلة الأولى وحدها تضيف ٣٠-٥٠% من التحسين، والمراحل مجتمعة تضيف أكثر من ٩٠%. كل مرحلة لها معايير نجاح قابلة للقياس.

# المنهجية والمعمارية المُستنتجة

## المنهجية المتبعة في الفحص

فُحصت الأدلة التالية فعليًا من الكود:

- المخطط الكامل لـ `src/db/schema.ts` و `drizzle/0000_gifted_rattler.sql` لفهم بنية الجداول والفهارس الموجودة والمفقودة.
- كل ملف **API route** ضمن `src/app/api/v1/` لتحليل استعلامات DB، أنماط الـ ORM، ومعالجة الأخطاء.
- كل ملف **مساعد (lib)** ضمن `src/lib/` (kinship, dedup, layout, treeFilter, ancestorFinder) لتحليل الخوارزميات وهياكل البيانات.
- الواجهة الخلفية الأساسية (Canvas)** ضمن `src/components/` لتحليل تخطيط العميل ومعالجة الـ state.
- إعدادات النشر (`ecosystem.config.cjs`، `deploy.sh`، `nginx.conf.example`) لاستنتاج بيئة التشغيل.

## البيئة التشغيلية المُستنتجة

| العنصر                | القيمة المُستنتجة                                      | الدليل                                                           |
|-----------------------|--------------------------------------------------------|------------------------------------------------------------------|
| نوع الاستضافة         | VPS Linux – Long-Running Node Process                  | <code>ecosystem.config.cjs:1-10</code> · <code>pm2 reload</code> |
| Reverse Proxy         | Nginx على نفس السيرفر                                  | <code>nginx.conf.example: proxy_pass 127.0.0.1:3001</code>       |
| قاعدة البيانات        | Supabase PostgreSQL عبر Pooler (المنفذ 6543)           | <code>src/db/index.ts:5,11</code>                                |
| Pooler Mode الـ       | Transaction-mode (PgBouncer) – له حد أقصى من الاتصالات | المنفذ 6543 (Transaction mode) vs 5432 (Session mode)            |
| عدد النسخ (instances) | ١ (افتراضي PM2) – لا توزيع عبء تلقائي                  | <code>ecosystem.config.cjs: instances: 1</code>                  |
| Realtime              | غير حرج (اشترك WebSocket واحد فقط في tree)             | <code>FamilyTreeCanvas.tsx:514-537</code>                        |

### 🕒 تأثير المعمارية على الخطة

كون التطبيق long-running process يفعل **ذاكرة تخزين محلية** (LRU Cache, React.cache) بشكل آمن، لكن الحد الأقصى من الاتصالات في Supabase Pooler يفرض سقفًا لعدد اتصالات DB الفعلية. كل توصية لاحقة تراعي هذين القيدين.

# ٣ جدول المشاكل الحرجة

ملخص جميع المشاكل المؤثرة في التقرير، مرتبة حسب الخطورة:

| # | المشكلة                                                                         | الملف / الأسطر                              | الخطورة | السيناريو                  |
|---|---------------------------------------------------------------------------------|---------------------------------------------|---------|----------------------------|
| ١ | تحميل كل سجلات<br>persons + relationships<br>marriages + في كل<br>طلب قراءة     | src/app/api/v1/tree/canvas/route.ts:56-58   | كارثي   | ٢٠ ألف سجل<br>Concurrent + |
| ٢ | عملية UPDATE داخل<br>GET request (تنظيف<br>base64)                              | src/app/api/v1/tree/canvas/route.ts:50-54   | كارثي   | Concurrent                 |
| ٣ | صفر فهرس (Index)<br>على جداول<br>relationships, persons,<br>marriages           | drizzle/0000_gifted_rattler.sql             | كارثي   | ٢٠ ألف سجل<br>Concurrent + |
| ٤ | Connection Pool بحجم<br>ه فقط (يكفي لـ ه طلبات<br>متزامنة)                      | src/db/index.ts:11-16                       | كارثي   | Concurrent                 |
| ٥ | تحميل كل السجلات +<br>Levenshtein على ٢٠ ألف<br>سجل عند كل إضافة<br>شخص         | src/app/api/v1/dedup/check/route.ts:23-24   | كارثي   | ٢٠ ألف سجل                 |
| ٦ | Levenshtein O(N×M)<br>على كامل الجدول مع<br>تخصيص مصفوفة لكل<br>استدعاء         | src/lib/dedup.ts:207-239                    | كارثي   | ٢٠ ألف سجل                 |
| ٧ | خوارزمية كشف الدورات<br>في النسب O(N²) بسبب<br>includes و Array.shift           | src/lib/kinship.ts:50-95                    | عالي    | ٢٠ ألف سجل                 |
| ٨ | خوارزمية dagre layout<br>في المتصفح على ٢٠ ألف<br>عقدة (main thread<br>blocked) | src/lib/layout.ts:100-260                   | عالي    | ٢٠ ألف سجل<br>(client)     |
| ٩ | إعادة معالجة ال layout<br>كاملة عند كل تغيير في<br>collapsed/filter             | src/components/FamilyTreeCanvas.tsx:500-512 | عالي    | ٢٠ ألف سجل<br>(client)     |

| #  | المشكلة                                                        | الملف / الأسطر                                     | الخطورة | السيناريو  |
|----|----------------------------------------------------------------|----------------------------------------------------|---------|------------|
| ١٠ | إرسال إيميلات المطالبات بشكل متسلسل for await                  | src/app/api/v1/claim/request/route.ts:102-131      | عالي    | Concurrent |
| ١١ | إدخال صف بصف (٢٠ ألف round-trip) في المزامنة الإدارية          | src/app/api/v1/admin/database/sync/route.ts:71-167 | عالي    | أي حجم     |
| ١٢ | إنشاء اتصال Postgres جديد في كل طلب إحصائيات                   | src/app/api/v1/admin/database/stats/route.ts:19-25 | عالي    | Concurrent |
| ١٣ | التحقق المزدوج من الهوية (Supabase Auth DB query +) بدون cache | src/lib/supabase/auth.ts:21-28                     | متوسط   | Concurrent |

### △ معنى الخطورة

**كارثي** = التطبيق ينهار أو يتعطل كليًا. **عالي** = تجربة المستخدم تتدهور بشكل محسوس (تأخير > ٥ ثوانٍ). **متوسط** = هدر موارد ولكن لا يكسر الوظيفة.

## ٤ السيناريو الأول: ٥٠-١٠٠ مستخدم متزامن

هذا السيناريو يفترض أن ٥٠ إلى ١٠٠ مستخدم يدخلون النظام في وقت واحد (مثل حملة إعلانية، إشعار بريد جماعي، أو نشر على وسائل التواصل). الفحص يكشف أن نقطة الانهيار موجودة في الكود الحالي، ولا تحتاج تجارب ضغط لتأكيدھا.

### ٤.١ حجم ال Connection Pool = ٥ فقط

كارثي

المشكلة ١: مجموعة اتصالات قاعدة البيانات صغيرة جدًا

الموقع: src/db/index.ts:11-16

المحرك: postgres-js

ال Pooler: Supabase Transaction-mode (max ~60 connection)

```
// src/db/index.ts:11-16 - الإعداد الحالي
export const client = postgres(connectionString, {
  max: 5, // فقط 5 اتصالات ⚠️
  idle_timeout: 20,
  connect_timeout: 10,
  max_lifetime: 60 * 10,
});
```

#### التأثير الفعلي

عند ١٠٠ طلب متزامن لـ `/api/v1/tree/canvas`، الطلبات من ٦ إلى ١٠٠ **تنتظر في طابور** للحصول على اتصال. كل طلب يقرأ ٣ جداول (persons + relationships + marriages) أي ٣ round-trips لل DB. ١٠٠ طلب x ٣ round-trips + ٥ اتصالات = **٦٠+ ثانية** في أسوأ الحالات، قبل حساب وقت المعالجة في Node.js نفسه.

#### التوصية

**زيادة max إلى ٢٠** مع تحديد `prepare: false` (مهم مع Supabase Transaction-mode Pooler). الحساب: ٤GB VPS يستهلك ٥٠٠MB لل Node، يبقى ~٣GB ⇒ ٢٠ اتصال x ١٠MB = ٢٠٠MB مريح.

### ٤.٢ كتابة داخل طلب قراءة

## كارثي

## المشكلة ٢: عملية UPDATE على الجدول الكامل عند كل طلب GET

الموقع:

src/app/api/v1/tree/canvas/route.ts:50-54

يُستدعى من: كل صفحات الشجرة (tree, /my-tree, /common-ancestor, /radial-tree/)

```
// السطور 50-54 - الإعداد الحالي
try {
  await db.update(personsTable)
    .set({ photo_url: null })
    .where(like(personsTable.photo_url, 'data:%'));
} catch { /* Safe catch */ }
```

## التأثير الفعلي

- ال UPDATE بدون LIMIT يطابق كل السجلات التي تبدأ بـ data .:
- يحدث عند كل طلب قراءة للشجرة.
- عند ١٠٠ طلب متزامن: ١٠٠ عملية UPDATE تنافس على نفس الجدول.
- لا يوجد فهرس على Full Table Scan + Row-level Lock ⇒ photo\_url لكل صف.

## التوصية

احذف الكود بالكامل من داخل ال route. وانقله إلى:

- Migration واحدة: ALTER TABLE persons ADD CONSTRAINT ... CHECK (photo\_url IS NULL OR photo\_url !~ '^data:')
- أو cron يومي: 0 3 \* \* \* ينفذ نفس ال UPDATE مرة واحدة.

## ٤.٣ لا فهارس قاعدة بيانات

كارثي

المشكلة ٣: صفر CREATE INDEX في ملف ال migration الكامل

الموقع: drizzle/0000\_gifted\_rattler.sql

ما هو موجود: PRIMARY KEY + UNIQUE + FOREIGN KEY فقط

المفقود: كل الفهارس على أعمدة البحث والتصفية

الفهارس المفقودة (مُستخرجة من أنماط الاستعلامات الفعلية)

| الجدول        | العمود             | يُستعلم في                                |
|---------------|--------------------|-------------------------------------------|
| relationships | person_id          | kinship.ts:9 · tree/canvas · admin/backup |
| relationships | related_person_id  | kinship.ts:25 · tree/canvas               |
| relationships | status             | tree/canvas:134,136,209 · dedup/check     |
| relationships | relationship_type  | tree/canvas:144,167,186 · kinship.ts:39   |
| relationships | created_by_user_id | admin/users/inspect:80                    |
| marriages     | husband_id         | tree/canvas:95 (مع N+1)                   |
| marriages     | wife_id            | tree/canvas:94 (مع N+1)                   |
| persons       | created_by_user_id | admin/users/inspect:86                    |
| login_logs    | created_at         | للأرشفة والتنقيح                          |

التوصية

استخدم CREATE INDEX CONCURRENTLY (بدون قفل الجدول) في migration جديد  
drizzle/0001\_performance\_indexes.sql . أضف أيضًا pg\_trgm GIN indexes على father\_name و first\_name  
لتسريع البحث التقريبي في dedup.

٤.٤ التحقق من الهوية (Auth)

## متوسط

## المشكلة ٤: round-trip مزدوج على كل طلب محمي

الموقع:

src/lib/supabase/auth.ts:21-28

عدد ال round-trips: ٢ لكل طلب (Supabase + DB)

```
// السطور 21-28
const supabase = createClient();
const { data: { user }, error: authError } = await
  supabase.auth.getUser(token); // استدعاء شبكة

const foundUsers = await db.select()
  .from(users)
  .where(eq(users.email, userEmail))
  .limit(1); // استعلام DB
```

## التأثير

على ١٠٠ طلب متزامن = ١٠٠ استدعاء API Supabase + ١٠٠ استعلام DB. لا يوجد caching داخل العملية (كل request يعيد التحقق من الصف).

الجزء الثاني ( `users.email` ) OK لأنه UNIQUE ⇒ فهرس موجود. لكن الجزء الأول يبقى bottleneck.

## التوصية

أضف `import { cache } from 'react'` حول الدالة `getAuthenticatedUser` لتفعيل **per-request deduplication**.  
وأضف `lru-cache` خارجي للتخزين المؤقت بين الطلبات (60 ~ TTL ثانية).

## ٤.٥ مشاكل متفرقة في التزامن

## عالي

## المشكلة ٥: إيميلات المطالبات تُرسل متسلسلة

الموقع:

src/app/api/v1/claim/request/route.ts:120-131

```
for (const steward of stewards) { // ⚠️ Loop متسلسل
  if (steward.email) {
    await sendEmailNotification({ /* ... */ });
  }
}
```

كل `sendEmailNotification` يستدعي SMTP. إذا كان هناك ٥ مشرفين و ١٠٠ مطالبة متزامنة =  $٥ \times ١٠٠ = ٥٠٠$  طلب SMTP متسلسلة. كل عملية SMTP تأخذ ٢s-٥ms ⇒ من ٤ دقائق إلى ١٦+ دقيقة.

عالي

## المشكلة هـ: اتصال Postgres جديد في كل طلب إحصائيات

الموقع: src/app/api/v1/admin/database/stats/route.ts:19-25

```
const cloudSql = postgres(cloudUrl); // ⚠️ اتصال جديد
let localSql: postgres.Sql | null = null;
try {
  localSql = postgres(localUrl, { connect_timeout: 3 }); // ⚠️ اتصال جديد
  await localSql`SELECT 1`;
}
```

لا يستخدم `client` المُصدّر من `@/db`. كل استدعاء = TCP handshake + TLS negotiation. على ١٠٠ طلب متزامن من صفحة admin = ٢٠٠ اتصال جديد، قد يصل إلى حد too many connections.

عالي

## المشكلة هـ: الإدخال صف بصف في المزامنة الإدارية

src/app/api/v1/admin/database/sync/route.ts:71-167

الموقع:

الأنماط المتأثرة: countries, users, persons, relationships, marriages, branch\_reviewers, merge\_requests

```
for (const p of srcPersons) { // ⚠️ 20,000 round-trips
  await targetSql`INSERT INTO persons (...) VALUES (...)`;
}
```

٢٠ ألف شخص = ٢٠ ألف round-trip منفصلة. كل round-trip على cloud DB = ٥٠-٢٠٠ms ⇒ المجموع ١٧ دقيقة إلى ٦٧+ دقيقة. يجب استخدام COPY أو INSERT ... VALUES ( ... ) دفعة واحدة.

# ه السيناريو الثاني: ٢٠ ألف سجل في الشجرة

يفترض هذا السيناريو امتلاء الشجرة ببيانات واقعية (~٢٠ ألف شخص، ~٤٠ ألف علاقة، ~١٥ ألف زواج). الفحص يكشف أن كل route تقريبًا يقوم بتحميل الجداول كاملة في الذاكرة، وأن الخوارزميات في الكود تعمل بتعقيد  $O(N^2)$  أو أعلى.

## ١. تحميل كل السجلات في كل طلب

كارثي

المشكلة ١: API يرجع كل البيانات دفعة واحدة

الموقع: src/app/api/v1/tree/canvas/route.ts:56-58

أيضًا في: dedup/check:23-24 · persons POST:377-378

// السطور 56-58

```
const dbPersons = await db.select().from(personsTable);
const dbRels = await db.select().from(relsTable);
const dbMarriages = await db.select().from(marriagesTable);
```

### الأرقام الفعلية مع ٢٠ ألف سجل

| البند                  | الحجم التقديري | التفصيل                        |
|------------------------|----------------|--------------------------------|
| Person record          | ~300 bytes     | مع biography و photo_url فارغة |
| persons 20,000         | ~6 MB          | JSON فقط                       |
| Relationship record    | ~200 bytes     | تقدير ٢ لكل شخص                |
| relationships 40,000   | ~8 MB          | JSON فقط                       |
| إجمالي الـ Response    | ~15-20 MB      | JSON عبر HTTP                  |
| ذاكرة Node لكل request | ~50-100 MB     | JS objects + Maps + Queues     |

### التأثير الفعلي

- استهلاك عرض النطاق: ١٠٠ مستخدم = ٢-١.٥ GB لكل تحديث.
- ذاكرة السيرفر: على Vercel/lambda بـ ١GB ⇒ ١٠ طلبات متزامنة تُسقط الـ instance.
- تجربة المستخدم: ٢٠-١٥ MB على 4G = ٣٠+ ثانية.
- معالجة إضافية: client يعيد التخطيط (راجع ٥.٤).

## ٥.٢ N+1 Query + Levenshtein على كامل الجدول

كارثي

### المشكلة ١: N+1 Query في معالجة الزيجات

الموقع: src/app/api/v1/tree/canvas/route.ts:93-95

```
dbMarriages.forEach((m) => {
  const wifeObj = dbPersons.find((p) => p.id === m.wife_id); // ⚠️ O(N)
  const husbandObj = dbPersons.find((p) => p.id === m.husband_id); // ⚠️ O(N)
  // ...
});
```

٢٠ ألف شخص + ١٥ ألف زواج = ٣٠ ألف استدعاء Array.find على مصفوفة بحجم ٢٠ ألف  $\Rightarrow$  ٦٠٠ مليون عملية مقارنة لكل طلب. على معالج ٣GHz  $\Rightarrow$  ٢٠٠-٥٠٠ms إضافية (وحدة المعالجة المركزية فقط).

كارثي

### المشكلة ٢: Levenshtein على كل شخص عند إضافة فرد جديد

الموقع: src/lib/dedup.ts:207-239 و src/app/api/v1/dedup/check/route.ts:23-24, 58-70

```
// src/lib/dedup.ts:207-239 - داخل calculateStringSimilarity
const track = Array(len2 + 1).fill(null).map(
  () => Array(len1 + 1).fill(0) // تخصيص في كل استدعاء
);
for (let j = 1; j <= len2; j += 1) {
  for (let i = 1; i <= len1; i += 1) { // ⚠️ O(len1 * len2)
    // ...
  }
}
```

#### ما يحدث فعلياً عند إضافة شخص واحد

1. تحميل كل الـ persons والـ relationships من DB.
2. تنفيذ findPotentialDuplicates على كل صف.
3. كل صف يُستدعى له calculateDeduplicationScore ٤ مرات (first, father, grand, family).
4. كل استدعاء Levenshtein يُنشئ مصفوفة ٢D.

المجموع على ٢٠ ألف سجل: ٤ مقارنات  $\times$  ٢٠ ألف شخص = ٨٠ ألف استدعاء Levenshtein عند كل إضافة شخص. كل استدعاء يُنشئ ٢٥-١٠٠ object/array  $\Rightarrow$  ضغط كبير على GC.

#### التقدير الزمني

وقت POST /api/v1/persons عند ٢٠ ألف سجل: من ٥ إلى ١٥ ثانية. عند كل إضافة شخص. تجربة المستخدم: سيئة جداً.

## ٥.٣ خوارزميات JS بتعقيد $O(N^2)$

عالي

## المشكلة ٣: Array.shift() و includes() في خوارزميات BFS

المواقع المتأثرة:

src/lib/layout.ts:148 · src/lib/kinship.ts:55, 81  
 · src/lib/treeFilter.ts:118, 143, 205, 345, 375  
 src/components/FamilyTreeCanvas.tsx:239

```
// النمط المتكرر في كل BFS:
while (queue.length > 0) {
  const current = queue.shift()!; // ⚠️ O(N) per shift
  // ...
}

// + result.includes(...) - O(N) per check
if (!result.includes(resolved.parentId)) result.push(resolved.parentId);
```

## التعقيد الفعلي

detectCircularLoop في kinship.ts:101-132 يقرأ كل العلاقات M في كل تكرار. مع ٢٠ ألف شخص و ٤٠ ألف علاقة:  
 فحص دورة نسبية واحدة = ١٠-٣٠ ثانية CPU. يحدث عند كل إضافة شخص (في persons/route.ts:412-419).

## البديل

```
// بمؤشر shift() استبدال
let head = 0;
while (head < queue.length) {
  const current = queue[head++]; // O(1)
}

// includes بـ Set.has استبدال
const seen = new Set();
if (!seen.has(resolved.parentId)) {
  seen.add(resolved.parentId);
  result.push(resolved.parentId);
}
```

٥.٤. انهيار المتصفح عند ٢٠ ألف عقدة

## على ٢٠ ألف عقدة في المتصفح Dagre layout: المشكلة ٤

عالي

الموقع: `src/lib/layout.ts:100-260` و `src/components/FamilyTreeCanvas.tsx:500-512`

## التأثير على العمل

- DOM + data structures في MB تستهلك ٢٠٠٠-٥٠٠ React Flow ذاكرة المتصفح: ٢٠ ألف عقدة.
- RADIAL ٣٠-٩٠ ثانية في وضع TB، وقت التخطيط: ٥-١٥ ثانية في وضع.
- **Main thread blocked:** `queue.shift()` يحجب الخيط الرئيسي.
- **Crash** GB RAM قد ينهار عند ٤ Chrome محتمل: تيويب.

## المشكلة المضاعفة: إعادة المعالجة عند كل تغيير

```
// FamilyTreeCanvas.tsx:500-512
useEffect(() => {
  if (rawNodesRef.current.length > 0) {
    applyGraphLayout(// ⚠️ يعيد كل الحسابات
      rawNodesRef.current, rawEdgesRef.current,
      currentLayout, collapsedNodes, activeFilter, ...
    );
  }
}, [currentLayout, collapsedNodes, activeFilter, applyGraphLayout]);
```

**wait time** = نفس الشيء. كل نقرة على شجرة كبيرة = TB↔BT أو تبديل الاتجاه **filter** فتح/إغلاق فرع واحد = إعادة تخطيط ٢٠ ألف عقدة. تغيير **time**.

# ٦ خطة الإصلاح المرحلية

خطة من ٦ مراحل متسلسلة، تبدأ بتغييرات صغيرة منخفضة المخاطر وتنتهي بإعادة هيكلة معمارية. كل مرحلة لها معايير نجاح قابلة للقياس.

| P0                                                  | P1                                               | P2                                                            | P3                                                           | P4                                                         | P5                                                    |
|-----------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------|
| <div><div>P0</div><div>Quick Wins · يوم</div></div> | <div><div>P1</div><div>Indexes · يوم</div></div> | <div><div>P2</div><div>Pagination + Cache · أسبوع</div></div> | <div><div>P3</div><div>Server Refactor · أسبوعان</div></div> | <div><div>P4</div><div>Client Optimize · أسبوع</div></div> | <div><div>P5</div><div>Monitoring · مستمر</div></div> |

**P0 Quick Wins – ربح فوري بدون مخاطر**

يوم واحد • ٤-٦ ساعات عمل

**الهدف:** تغييرات صغيرة بدون مخاطر، تحصد ٣٠-٥٠٪ من التحسين.**P0.1 tree/canvas في GET داخل UPDATE إزالة**

الملف: tree/canvas/route.ts:50-54

الجهد: ١٥ دقيقة

المخاطرة: صفر

يومي cron واحد أو migration بالكامل، وانقل التنظيف إلى try/catch احذف كتلة الـ

**P0.2 بذكاء Connection Pool زيادة**

الملف: src/db/index.ts:11-16

الجهد: ١٠ دقائق

المخاطرة: منخفضة

(Supabase Transaction-mode Pooler مهم مع prepare: false و max: 20 اضبط).

**P0.3 queue.shift() بمؤشر استبدال**

الملفات: kinship.ts, layout.ts, treeFilter.ts, FamilyTreeCanvas.tsx

الجهد: ٣٠ دقيقة

الربح:  $O(N^2) \rightarrow O(N)$ 

نمط موحد:

```
let head = 0;
while (head < queue.length) {
  const current = queue[head++]; //  $O(1)$  بدلاً من  $O(N)$ 
  // ...
}
```

**P0.4 includes() بـ Set.has() استبدال**

الملف: src/lib/kinship.ts:13, 28

الجهد: ١٥ دقيقة

الربح:  $O(N^2) \rightarrow O(N)$ **P0.5 افتراضياً supabase channel تعطيل**

الملف:

src/components/FamilyTreeCanvas.tsx:514-537

الجهد: ٢٠ دقيقة

البديل: polling كل ٦٠ ثانية (feature flag)

كامل عند أي تغيير re-fetch خفيف لتجنب polling بـ WebSocket حساسة (مؤكد من المستخدم). استبدل realtime البيانات ليست

**✓ P0 معيار نجاح**

٨٠٠ (مستخدم واحد) من GET /api/v1/tree/canvas ms إلى ٢٠٠ ms CPU استخدام. VPS على ٤٠ أقل ٪.

## P1 Database Indexes – أعمق ربح من حيث الجهد

يوم واحد • يوم كامل

هذه المرحلة وحدها قد تفي بالسيناريو الأول. Seq Scan بدلاً من Index Scan الهدف: كل استعلام يصبح

### P1.1 Migration 0001 جديد\_performance\_indexes.sql

الجهد: ٣ ساعات

(CONCURRENTLY) المخاطرة: منخفضة جدًا

```
-- الفهارس الحرجة على relationships
CREATE INDEX CONCURRENTLY idx_relationships_person_id ON relationships(person_id);
CREATE INDEX CONCURRENTLY idx_relationships_related_id ON relationships(related_person_id);
CREATE INDEX CONCURRENTLY idx_relationships_status ON relationships(status);
CREATE INDEX CONCURRENTLY idx_relationships_type ON relationships(relationship_type);
CREATE INDEX CONCURRENTLY idx_relationships_person_status
ON relationships(person_id, status) WHERE status IN ('VERIFIED', 'PENDING');

-- الفهارس marriages
CREATE INDEX CONCURRENTLY idx_marriages_husband ON marriages(husband_id);
CREATE INDEX CONCURRENTLY idx_marriages_wife ON marriages(wife_id);

-- فهارس persons
CREATE INDEX CONCURRENTLY idx_persons_country ON persons(country_id) WHERE country_id IS NOT NULL;
CREATE INDEX CONCURRENTLY idx_persons_created_by ON persons(created_by_user_id);
CREATE INDEX CONCURRENTLY idx_persons_claim_status ON persons(claim_status) WHERE claim_status IS NOT NULL;
```

### P1.2 للبحث التقريبي pg\_trgm إضافة

الجهد: ٣٠ دقيقة

P3.3 يحل جزءًا من

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX CONCURRENTLY idx_persons_first_name_trgm
ON persons USING gin (first_name gin_trgm_ops);
CREATE INDEX CONCURRENTLY idx_persons_father_name_trgm
ON persons USING gin (father_name gin_trgm_ops);
CREATE INDEX CONCURRENTLY idx_persons_family_name_trgm
ON persons USING gin (family_name gin_trgm_ops);
```

#### ✓ P1 معيار نجاح

وقت dedup/check على ٢٠ ألف سجل: من ١٥-٥٠. Seq Scan بدلاً من Index Scan يُظهر EXPLAIN ANALYZE ms. ثنائية إلى ٢٠٠-٥٠.

## P2 API Pagination + Caching – تقليل payload

أسبوع ٠ - ٣ - ٤ أيام

pagination + viewport filter. عبر KB إلى ٥٠٠ MB من ١٥-٢٠ payload الهدف: تقليل حجم

### P2.1 أصغر endpoint إلى tree/canvas تقسيم

الجهود: ١ يوم

- `GET /api/v1/tree/canvas` – فقط batch البيانات الوصفية + أول
- `GET /api/v1/tree/canvas/batch?index=0&size=5000` – batch إضافي
- `GET /api/v1/tree/person/:id` (lazy) تفاصيل شخص واحد –

### P2.2 في طبقة الخادم LRU Cache

الملف الجديد: `src/lib/cache.ts`

الجهود: ١ يوم

```
import { LRUCache } from 'lru-cache';

const treeCache = new LRUCache<string, any>({
  max: 100, // 100 viewport = ~50MB max
  ttl: 1000 * 60 * 5, // دقائق 5
  allowStale: true,
});
```

PM2 VPS = الـ cache عند rebuild (OK، reload pm2 يعيش في الذاكرة. عند cache

### P2.3 request للطلبات المكررة داخل نفس الـ React.cache()

الملف: `src/lib/supabase/auth.ts`

الجهود: ٣٠ دقيقة

#### ✓ P2 معيار نجاح

ms (cached). وقت استجابة: من ٢-٥ ثوانٍ إلى ١٠٠-٣٠٠ KB. إلى ٥٠٠ MB من ١٥-٢٠ payload: حجم

## P3 Server Refactoring – نقل المنطق لـ PostgreSQL

أسبوعان • ٨-١٠ أيام

استبدال خوارزميات PostgreSQL Recursive CTE + Materialized Views بـ JS: الهدف:

### P3.1 – استبدال kinship.ts بـ PostgreSQL Recursive CTE

الملف الجديد: src/db/queries/kinship.ts

```
-- JS في detectCircularLoop بدلاً من
WITH RECURSIVE descendants AS (
  SELECT person_id, related_person_id
  FROM relationships
  WHERE person_id = $1 AND relationship_type = 'PARENT'
  UNION ALL
  SELECT r.person_id, r.related_person_id
  FROM relationships r
  INNER JOIN descendants d ON r.person_id = d.related_person_id
  WHERE r.relationship_type = 'PARENT'
)
SELECT 1 FROM descendants WHERE related_person_id = $2 LIMIT 1;
```

### P3.2 – Materialized View للأشجار

(refresh لل trigger بما فيها) الجهد: ٢ أيام

```
CREATE MATERIALIZED VIEW mv_tree_full AS
SELECT p.id, p.first_name, p.father_name, p.grand_father_name, p.family_name,
       p.gender, p.birth_year, p.is_alive, p.is_placeholder,
       r.id as rel_id, r.person_id, r.related_person_id,
       r.relationship_type, r.status as rel_status
FROM persons p
LEFT JOIN relationships r
  ON (r.person_id = p.id OR r.related_person_id = p.id)
  AND r.status IN ('VERIFIED', 'PENDING')
WHERE p.id IN (
  SELECT DISTINCT person_id FROM relationships
  UNION SELECT DISTINCT related_person_id FROM relationships
);
```

### P3.3 – بمؤهلات سريعة dedup/check تجزئة

الملف: src/lib/dedup.ts + dedup/check/route.ts

: (بدلاً من ٢٠ ألف) top 50 فقط على Levenshtein للترتيب، ثم pg\_trgm استخدم

```
SELECT id, first_name, father_name, grand_father_name, family_name
FROM persons
WHERE similarity(first_name, $1) > 0.3
  OR similarity(father_name, $2) > 0.3
ORDER BY GREATEST(
  similarity(first_name, $1),
```

```
similarity(father_name, $2)
) DESC
LIMIT 50;
```

أسرع X إلى ٢٠٠ استدعاء = ٤٠٠٠ Levenshtein الريح: من ٨٠ ألف استدعاء

### P3.4 – bulk insert بـ admin/database/sync إصلاح

الملف: src/app/api/v1/admin/database/sync/route.ts:71-167

```
// 20,000 round-trips قبل:
for (const p of srcPersons) {
  await targetSql`INSERT INTO persons (...) VALUES (...)`;
}

// واحد round-trip بعد:
await targetSql`INSERT INTO persons (id, first_name, ..., created_at)
VALUES ${srcPersons.map(p => sql`${p.id}, ${p.first_name}, ..., ${p.created_at}`)}`;
```

#### ✓ P3 معيار نجاح

- POST /api/v1/persons : ٥٠٠-٢٠٠ إلى ١٥-٥ ثانية ms
- dedup/check : ٢٠٠-٥٠ إلى ٥-٣ ثواني ms
- admin/database/sync لـ ٢٠ ألف سجل: من ٦٧ دقيقة إلى ١٠-٣٠ ثانية

## P4 منع انهيار المتصفح – Client Optimization

أسبوع ٠ ٣-٤ أيام

تشغيل Dagre في Web Worker + Viewport Culling + Code Splitting. الهدف:

### P4.1 – Web Worker إلى Dagre layout نقل

الملف الجديد: src/lib/workers/layout.worker.ts

الريخ: Main thread ٦٠ ⇒ ٦٠fps

```
// FamilyTreeCanvas.tsx
const layoutWorker = new Worker(
  new URL('../lib/workers/layout.worker.ts', import.meta.url)
);
layoutWorker.postMessage({ nodes, edges, options });
layoutWorker.onmessage = (e) => setNodes(e.data.nodes);
```

### P4.2 – React Flow Viewport Culling

الملف: src/components/FamilyTreeCanvas.tsx

يحتوي ١٠٠-٢٠٠ عقدة بدلاً من ٢٠ ألف DOM: الريح

```
const visibleNodes = useMemo(() =>
  nodes.filter(n => isInViewport(n.position, reactFlowInstance.getViewport()))
, [nodes, viewport]);
```

### P4.3 – Code Splitting لكل صفحة

الملفات: src/app/\*/page.tsx

RadialTreeSVG، LayoutToolbar، وModal. للمكونات الثقيلة dynamic(() => import(...)) تأكد من استخدام

#### ✓ P4 معيار نجاح

ms. إلى > ٢٠٠ ms+ INP: ٢٠٠٠. وقت الفتح الأول لـ /tree على ٢٠ ألف سجل: من ٦٠+ ثانية إلى ٢-٥ ثوانٍ

**P5** **Monitoring + Hardening – قياس مستمر**

مستمر • ٢-٣ أيام للإعداد

تنبيهات على التراجع + Core Web Vitals الهدف: قياس الـ

**P5.1 – web-vitals إضافة**

الهدف: الملف الجديد: src/lib/vitals.ts

```
import { onLCP, onINP, onCLS } from 'web-vitals';
// timeseries DB داخلي للتخزين في endpoint أرسل إلى
```

**P5.2 – Middleware لتسجيل الاستعلامات البطيئة**

الهدف: نصف يوم

```
const start = Date.now();
// ... handler
if (Date.now() - start > 1000) {
  console.warn({ route, duration: Date.now() - start });
}
```

**P5.3 – Pool حماية الـ + تنبيهات**

- إذا TTFB > 2s إيميل/تنبيه إذا `/tree/canvas`
- (قريب من الحد) `pg_stat_activity.count > 15` إيميل إذا

## الجدول الزمني والنتائج المتوقعة

### الجدول الزمني المقترح

| المخرج                                 | المرحلة                 | الأسبوع            |
|----------------------------------------|-------------------------|--------------------|
| (ms إلى ٢٠٠ ms من ٨٠٠ TTFB) نشر + قياس | Quick Wins P0           | الأسبوع ١، يوم ١-٢ |
| تقرير + maintenance window تشغيل في    | Indexes P1              | الأسبوع ١، يوم ٣-٤ |
| تقرير أسبوع ١ - قابل للمشاركة          | قياس + تقرير            | الأسبوع ١، يوم ٥   |
| feature flag نشر مرحلي مع              | Pagination + Caching P2 | الأسبوع ٢-٣        |
| A/B عمل في فرع منفصل + اختبار          | Server Refactoring P3   | الأسبوع ٤-٥        |
| على متصفحات حقيقية A/B اختبار          | Client Optimization P4  | الأسبوع ٦          |
| لوحة مراقبة + تنبيهات                  | Monitoring P5           | +الأسبوع ٧         |

### النتائج المتوقعة بالأرقام

:التوقعات تستند إلى أحجام البيانات الفعلية في الكود والبنية المستنتجة

| P3-P4 بعد         | P0-P2 بعد   | اليوم (الوضع الحالي)    | المقياس                            |
|-------------------|-------------|-------------------------|------------------------------------|
| ٢٠٠-١٠٠ms         | ٣٠٠-١٠٠ms   | ٥-٢ ثوانٍ               | على /tree/canvas (٢٠ ألف سجل) TTFB |
| ٣٠٠-٢٠٠ms         | ٥٠٠-٢٠٠ms   | (timeout) +٦٠ ثانية     | (١٠٠ مستخدم متزامن) TTFB           |
| ٥٠٠-٢٠٠ms         | ٢-١ ثانية   | ١٥-٥ ثانية              | إضافة شخص جديد (٢٠ ألف سجل)        |
| ٥-٢ ثوانٍ         | ٢٠-١٠ ثانية | Tab crash + ٩٠-٣٠ ثانية | على المتصفح (٢٠ ألف سجل) Layout    |
| ٢٠٠ KB (viewport) | ٥٠٠ KB      | ٢٠-١٥ MB                | payload حجم /tree/canvas           |
| ٢٠٠-٥٠ms          | ١ ثانية     | ٥-٣ ثوانٍ               | (٢٠ ألف سجل) dedup/check           |
| ٣٠-١٠ ثانية       | بدون تغيير  | ٦٧ دقيقة                | (٢٠ ألف سجل) admin/database/sync   |

### (Lighthouse والـ Vercel من قياس) المقاييس المستهدفة

| سبب       | يحتاج تحسين | جيد (الهدف)   | Core Web Vital                 |
|-----------|-------------|---------------|--------------------------------|
| ٤ ثوانٍ > | ٤ ثوانٍ ≤   | ٢.٥ ثانية ≤ ✓ | LCP (Largest Contentful Paint) |

| سئي     | يحتاج تحسين | جيد (الهدف) | Core Web Vital                  |
|---------|-------------|-------------|---------------------------------|
| > ٥٠٠ms | ≤ ٥٠٠ms     | ≤ ٢٠٠ms ✓   | INP (Interaction to Next Paint) |
| > ٠.٢٥  | ≤ ٠.٢٥      | ≤ ٠.١ ✓     | CLS (Cumulative Layout Shift)   |

## ٨ الخلاصة والتوصيات

### الحكم المعماري

لكن الفحص يكشف أنه يحتوي على (Next.js + Drizzle + Supabase) مبني بأسلوب وظيفي مقبول في فئته **FamilyTree Al-Nammari** التطبيق (تحميل) ١٣ نمطًا معماريًا ضارًا بالأداء، بعضها يصل إلى درجة كارثية عند الحجم المستهدف. النماذج الإشكالية ليست أخطاءً منعزلة، بل هي اختيارات تصميمية متكررة (عدم وجود فهرس JS، في  $O(N^2)$  الجداول كاملة، خوارزميات

### المسارات المقترحة للبدء

فورًا  $P0 + P1$  • الخيار ١ – موصى به

DB. من كل طلب قراءة، ويضاعف سرعة كل استعلام round-trips يومين عمل فقط، تأثير كبير، مخاطرة صفر. يزيل ٣

مما  $P2 + P3$  – الخيار ٢

DB. قريب للمنتج. يجلب الحل الجذري لـ deadline أسبوعان، مناسب إذا عندك

$P2$  ثم  $P3$  – الخيار ٣

أولاً قبل أي تحسينات أخرى DB المتوقع ضخم جدًا (آلاف مترامين) وتريد الحل الجذري للـ DAU ثلاثة أسابيع، مناسب إذا الـ

### مبادئ الإصلاح

. قس قبل أن تُصلح: كل مرحلة لها معيار نجاح قابل للقياس. لا تحتفظ بتغيير لم يُحسن الأرقام.

. غير شيء واحد في كل مرة: إذا اشتغلت ٣ تحسينات معًا، لن تستطيع أن تعرف أيها أفاد.

. لتفادي تكرارها PERF.md اجعلها في – git history سجل المحاولات الفاشلة: التعديلات المرجعة تترك أثرًا في الـ

. ليس إنجاز، regression هو validation حافظ على الصحة الوظيفية: الـ "تحسين" الذي يربح بتخطي

### ما لا يجب فعله

. في كل مكان – تكرار النمط بدون قياس يُضيف تعقيدًا بدون ربح React.memo لا تضيف

. الخوارزميات المتقدمة بدون فهرس = بطء مختلف فقط –  $P0+P1$  قبل  $P3$  لا تبدأ بـ

. سيفشل بصمت – transaction داخل Drizzle migrations CONCURRENTLY لا تستخدم

.  $P2$  في route.ts:8 ضروري للأمان – نُقله في force-dynamic لا تفترض أن

### الخطوة التالية

سيكون PR جاهز للتنفيذ مع كل التغييرات والاختبارات، وأرفعه كفرع منفصل للمراجعة. كل PR بعد موافقتك على خيار من الثلاثة، أجهز: مُقترنًا بـ

- (Lighthouse + قياس TTFB) اختبار أداء قبل/بعد
- بالأرقام (معياري النجاح أو سبب الإرجاع) commit تقرير

- في حالة الطوارئ rollback دليل الـ

تم إعداد هذا التقرير بناءً على فحص فعلي للكود في المستودع بتاريخ ٦ سبتمبر ٢٠٢٦. كل ادعاء مرفق بـ `file:line`. لا توجد تكهنات أو تقديرات غير مبنية على الكود.